

Estruturas de Repetição

Para fins de praticidade, o documento a seguir explicará o conceito de “estruturas de repetição” exclusivamente na linguagem python. No entanto, a escolha da linguagem fica a critério do programador, mas a lógica usada pode ser aplicada a qualquer linguagem de programação com sintaxe diferente.

O que são?

Estruturas de repetição, ou loops, são recursos da linguagem que executam um mesmo bloco de código várias vezes, sem que você precise reescrevê-lo. O Python tem dois tipos: o `for`, que percorre item a item um iterável (lista, string, `range()`), e o `while`, que repete enquanto uma condição for verdadeira. São a base de tarefas como processar listas, ler arquivos e validar entradas.

Loops utilizando “while”

O comando `while` executa um loops através de uma condição estabelecida no código. Geralmente se utiliza este comando quando **não se sabe quantas interações irá precisar**. Em outras palavras:

“Só saia da estrutura de repetição quando a condição não for mais satisfeita”

Exemplo:

```
1. contador = 0
2.
3. while contador < 5:
4.     print(f'Valor do contador é {contador}')
5.     contador += 1
```

Nesse exemplo vemos acontecer uma soma simples. Funciona da seguinte forma: Enquanto - `while` em inglês, o “contador” for menor que 5, será acrescentado 1 ao “contador”, e ainda por diante até que o valor seja um número. Então, quando ele alcançar o valor 5, a condição `contador < 5` não será mais satisfeita, finalizando o loop `while`! Veja o que é exibido:

```
Valor do contador é 1
Valor do contador é 2
Valor do contador é 3
Valor do contador é 4
```

Break e Continue

Existem duas palavras reservadas da linguagem que servem para auxiliar no controle do fluxo da estrutura de repetição. São elas: `break` e o `continue`! Utilizamos o `break` para parar a execução de um loop. Geralmente utilizamos uma estrutura condicional com `if` para então usar a cláusula `break`! Já o `continue` é utilizado para pular todo código que estiver após esta cláusula, levando em frente na próxima iteração do loop!

Loops utilizando `while` e `else`

Podemos ainda adicionar a cláusula `else` em loops `while`!

O `else` nos possibilita executar um bloco de código após a condição ter sido satisfeita.

Porém, o `else` não é executado quando o `while` encontra uma cláusula `break`!

Vamos entender melhor no exemplo:

```
1. while True:
2.     tentativas = 0
3.     numero_magico = 3
4.     numero = int(input('Adivinhe o número mágico
5. (0 a 100): '))
6.
7.     if numero == numero_magico:
8.         print('Corre pra Loteria! Hoje é seu dia de sorte
9. *.*')
10.        break
11.        tentativas += 1
12.    else:
13.        print('Teeeeente outra veeeee xD')
```

Nesse exemplo, perguntamos um número ao usuário e ele deve acertar o número escolhido em 3 tentativas.

Se ele acertar, o texto “Corre pra Loteria! Hoje é seu dia de sorte *.*” deve ser mostrado. Caso contrário, se ele não acertar em 3 tentativas, o seguinte texto deverá ser mostrado: “Teeeeente outra veeeee xD” e o loop será repetido.

Perceba que o `else` não deve ser executado caso o código passe pela cláusula `break`!

O comando For

O for em Python é usado para percorrer um iterável (lista, tupla, string, dicionário) e executar um bloco de código uma vez para cada item. Geralmente se utiliza o for para percorrer todos os itens de uma lista.

Vamos ver um exemplo prático:

```
lista = [1, 2, 3, 4, 5]

for item in lista:
    print(item)
```

Vamos entender passo a passo:

Na primeira iteração, o item vai receber o valor do primeiro elemento da lista lista, que é 1. Portanto, print(item) vai mostrar o valor 1, na segunda o valor 2, e assim por diante.

Adicionar o *else* ao final do for nos possibilita executar um bloco de código após o iterável ter sido completamente percorrido.

Veja um exemplo:

```
for item in sequencia:
    print(item)
else:
    print('Todos os itens foram exibidos com sucesso')
```

Algumas utilidades com o for

- Separar caracteres de um *string*. Como usar o comando for in palavra para percorrer cada letra.

```
for caractere in 'Python':
    print(caractere)
```

A saída será cada letra separada por linha:

```
...P  
y  
t  
h  
o  
n
```

- Repetição de operações. Pegando uma range junto de uma operação de multiplicação para exibir todos os valores de uma determinada tabuada.

```
for i in range(1, 11):  
    mul = i*2  
    print(f"2x{i}={mul}")
```

Saída será uma tabuada com base dois:

```
...2x1=2  
2x2=4  
2x3=6  
2x4=8  
2x5=10  
2x6=12  
2x7=14  
2x8=16  
2x9=18  
2x10=20
```

- Percorrer todos os itens de uma *lista*. Como pegar cada item de uma lista, “Processador”, “Teclado”, e “Mouse”, e usar o comando "for item in computador", para percorrer cada nome.

```
computador = ['Processador',  
              'Teclado', 'Mouse']  
  
for item in computador:  
    print(item)
```

A saída será a impressão de cada item:

```
...Processador  
Teclado  
Mouse
```

Fonte:

<https://pythonacademy.com.br/blog/estruturas-de-repeticao>

<https://pythonacademy.com.br/blog/loops-com-while-no-python>